LENGUAJES II · CUARTO SEMESTRE

TecnoSalta

Tienda de componentes de PC hecha en Java con JavaFX

[Corregidor Martin Guillermo · Flores C. José M · Fuentes Facundo · Tapia Lucas Fabricio Camacho Daniel

[Lucero Nancy del Carmen] · [12/11/2026]

EXPOSITOR 1 · EL PROYECTO

Qué hace la tienda

Cliente

- Se registra e inicia sesión
- Recorre el catálogo y filtra por nombre, categoría y precio
- Arma el carrito y confirma la compra
- Consulta el historial de sus pedidos

Administrador

- Da de alta, modifica y da de baja productos
- Gestiona las categorías
- Controla el stock y la foto de cada producto
- Ve todos los pedidos y les cambia el estado

La compra es simulada: descuenta stock y genera el pedido, pero no hay pagos reales, envíos ni correos.

Objetivos

OBJETIVO GENERAL

Desarrollar una tienda de e-commerce completa en Java que aplique programación orientada a objetos y una arquitectura en capas.

Modelar

El dominio con clases, enumeraciones y relaciones UML

Separar

La aplicación en capas con una sola responsabilidad cada una

Construir

Una interfaz gráfica con JavaFX, FXML y CSS

Guardar

Los datos entre ejecuciones sin depender de un servidor

Validar

Las reglas del negocio: stock, precios y contraseñas

Probar

La lógica de la compra con pruebas automáticas JUnit

EXPOSITOR 1 · EL PROYECTO

Qué se implementó

8
pantallas en FXML

37
archivos Java

12
pruebas JUnit que pasan

17
componentes de ejemplo

Cliente

- Registro e inicio de sesión
- Catálogo con tres filtros
- Detalle y carrito
- Compra e historial

Administrador

- Productos con foto y stock
- Categorías
- Estado de los pedidos
- Cancelar devuelve el stock

En todo el programa

- Contraseñas con BCrypt
- Datos en tienda.dat
- Modo claro y oscuro
- Ejecutable sin instalar Java

EXPOSITOR 2 · DISEÑO

Arquitectura general en capas



Modelo (Usuario, Producto, Pedido, Carrito...): lo usan todas las capas.

La regla es una sola: cada capa llama únicamente a la siguiente. Así cada parte se puede entender, cambiar y probar por separado.

EXPOSITOR 2 · DISEÑO

Librerías y herramientas

JavaFX
La interfaz gráfica: ventanas, botones y tablas. Cada pantalla es un archivo FXML

AtlantaFX
Tema visual moderno, con modo claro y modo oscuro

jBCrypt
Guarda las contraseñas cifradas, nunca como texto

JUnit 5
Pruebas automáticas de la lógica del carrito y la compra

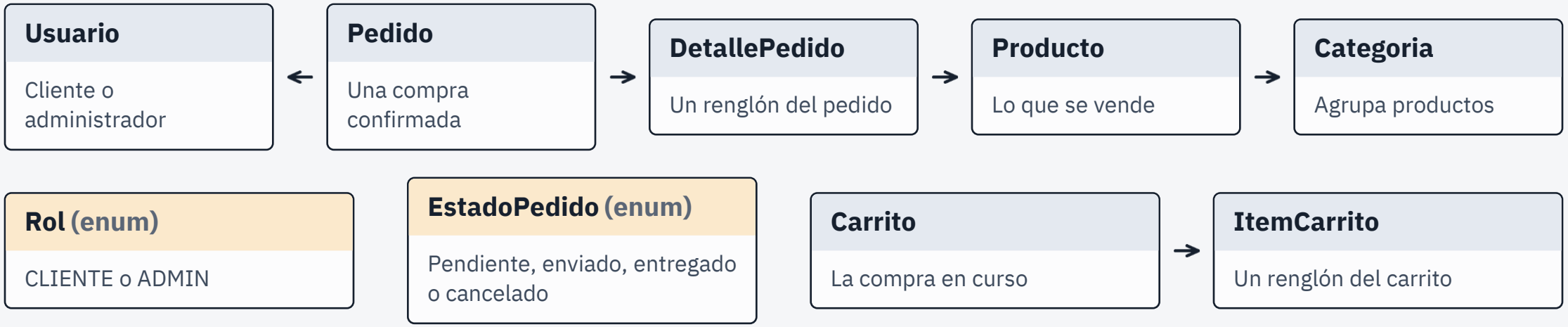
Maven
Descarga las librerías, compila, prueba y ejecuta

Java 25
El lenguaje de todo el proyecto, en su versión con soporte largo

Pocas librerías a propósito: el código sigue siendo Java claro y fácil de explicar.

EXPOSITOR 2 · DISEÑO

Diagrama de clases UML



- Un usuario hace muchos pedidos; cada pedido tiene uno o más renglones, y cada renglón apunta a un producto
- El carrito y el pedido se parecen, pero el carrito usa el precio actual y el pedido guarda el precio pagado

EXPOSITOR 2 · DISEÑO

Cómo se guardan los datos

En memoria

Mientras el programa corre, todo vive en listas de Java



tienda.dat

Cada cambio se guarda en este archivo y al abrir se vuelve a leer

Datos de ejemplo

La primera vez se cargan solos 2 usuarios y 17 componentes

- Un producto que ya se vendió no se borra: se oculta del catálogo
- El pedido guarda el precio pagado, aunque después cambie
- El carrito no se guarda: dura lo que dura la sesión

EXPOSITOR 3 · LOGIN Y CATÁLOGO

Login y registro



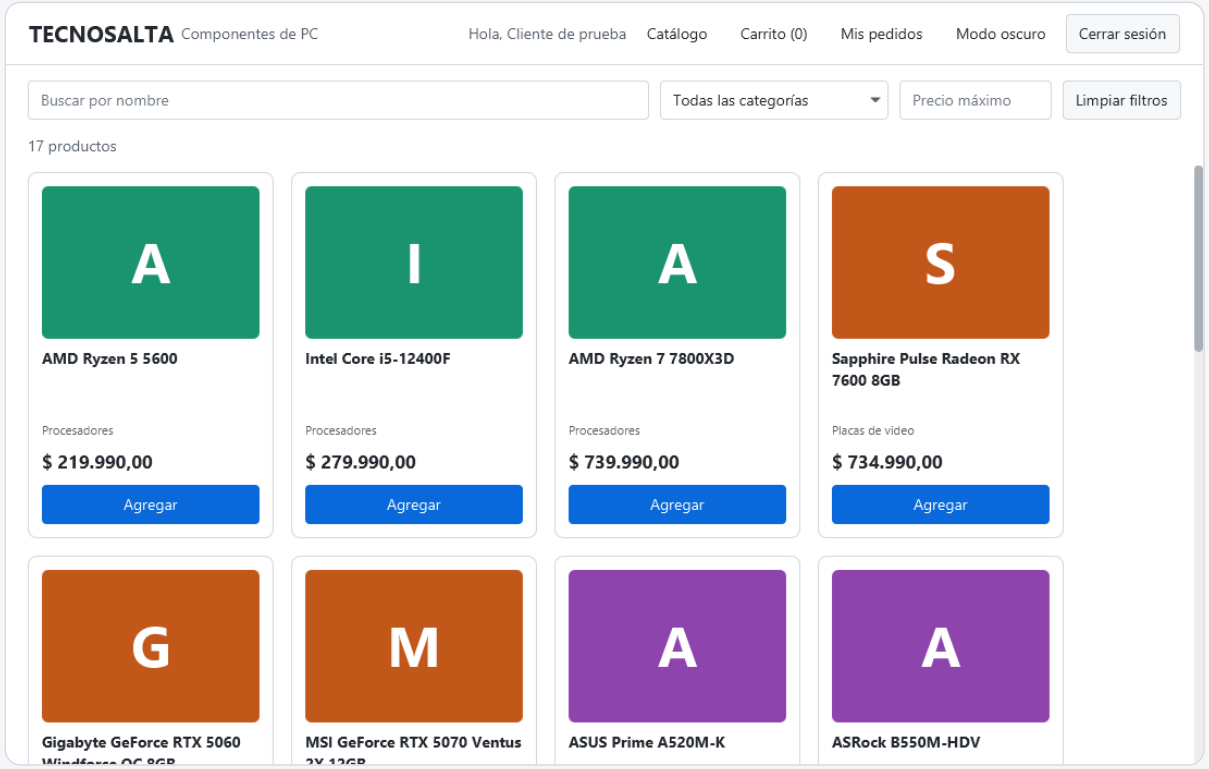
Qué hace el código

- LoginController toma el email y la contraseña y llama a AuthService
- AuthService compara la contraseña con el hash guardado usando BCrypt
- Si coincide, guarda el usuario en Sesion y abre el catálogo o el panel según el rol
- El registro valida nombre, email y una contraseña de 6 caracteres o más

`LoginController` · `RegistroController` · `AuthService` · `Sesion`

EXPOSITOR 3 · LOGIN Y CATÁLOGO

Catálogo con búsqueda y filtros



Qué hace el código

- Cada producto es una tarjeta dentro de un FlowPane con desplazamiento
- Al escribir o cambiar un filtro, el controlador vuelve a pedir la lista al servicio
- Sin unidades, el botón se deshabilita y dice “Sin stock”
- Un clic en la tarjeta abre el detalle, con descripción y cantidad

`CatalogoController` · `DetalleController` · `ProductoService`

Cómo responde el catálogo

1
El usuario escribe o elige un filtro

2
El controlador detecta el cambio

3
ProductoService decide qué productos coinciden

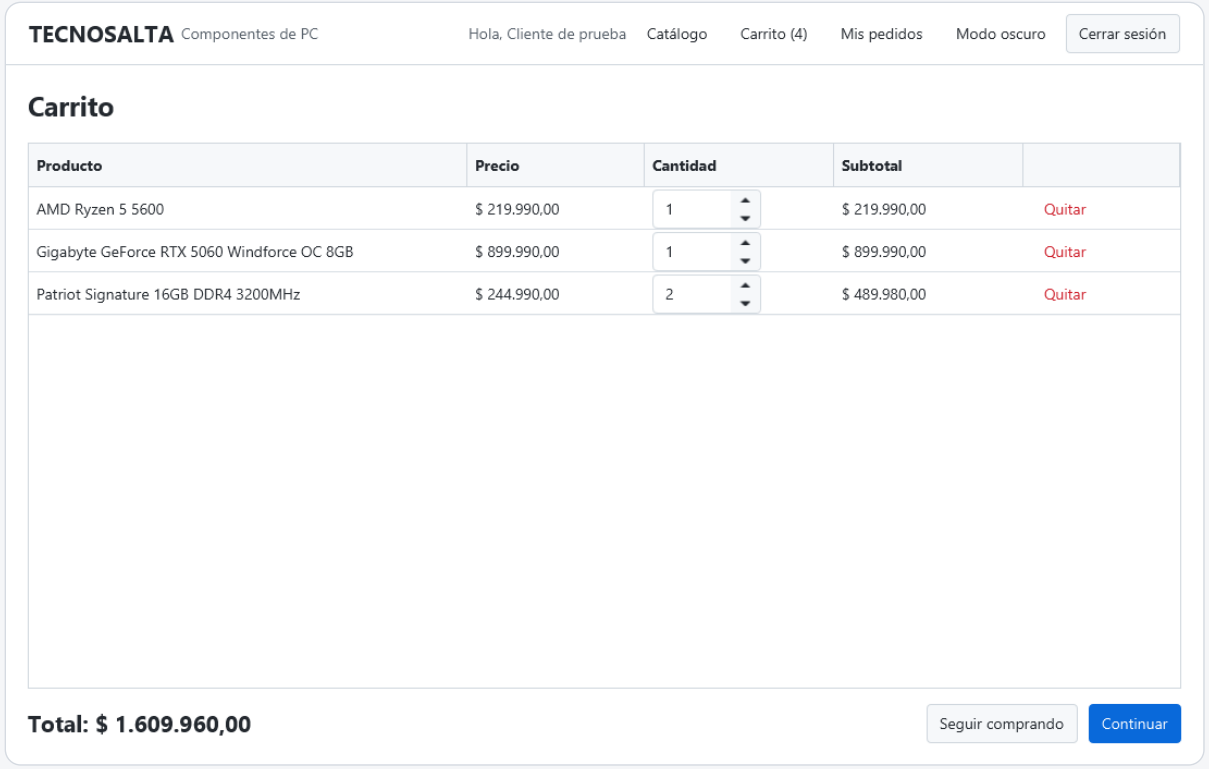
4
Se vuelven a dibujar las tarjetas

```
// Un producto se muestra si cumple los tres filtros
if (coincideNombre && coincideCategoria && coincidePrecio) {
    resultado.add(producto);
}
```

Un filtro vacío no filtra. La decisión está en el servicio y no en la pantalla, por eso se puede probar sin abrir ventanas.

EXPOSITOR 4 · CARRITO Y COMPRA

Carrito de compras



Qué hace el código

- El carrito vive en la Sesion: no se guarda en el archivo
- CarritoService valida el stock antes de agregar o cambiar una cantidad
- Si no alcanza, lanza StockInsuficienteException, una excepción propia
- Cada renglón tiene un Spinner que va de 1 al stock del producto

`CarritoController · CarritoService · Carrito · ItemCarrito`

EXPOSITOR 4 · CARRITO Y COMPRA

La compra es todo o nada

1. Verificar

Se revisa el stock de todo el carrito. Si falta algo, se avisa y no se cambia nada

→

2. Registrar

Se crea el pedido con el precio de cada producto y se descuenta el stock

→

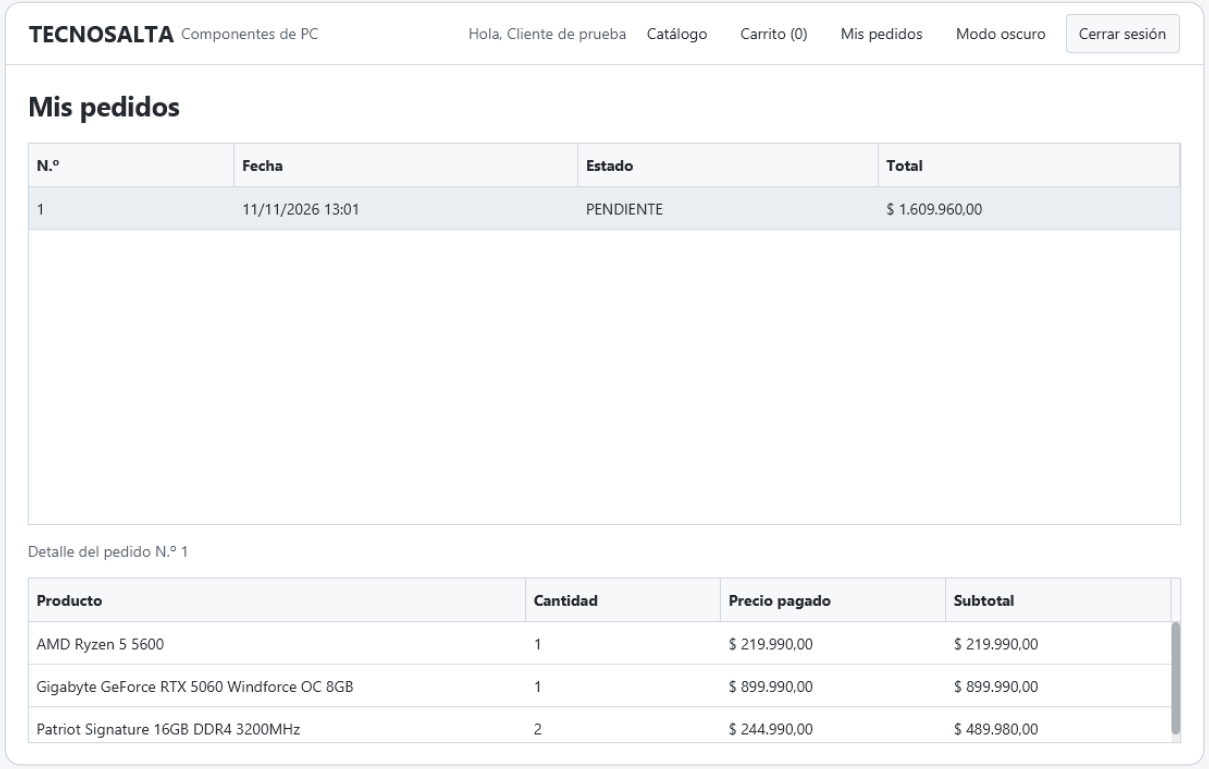
3. Guardar

Se guarda todo junto en el archivo y se vacía el carrito

Si falta stock se lanza **StockInsuficienteException**, una excepción propia: nunca queda un pedido a medias.

EXPOSITOR 4 · CARRITO Y COMPRA

Historial de pedidos



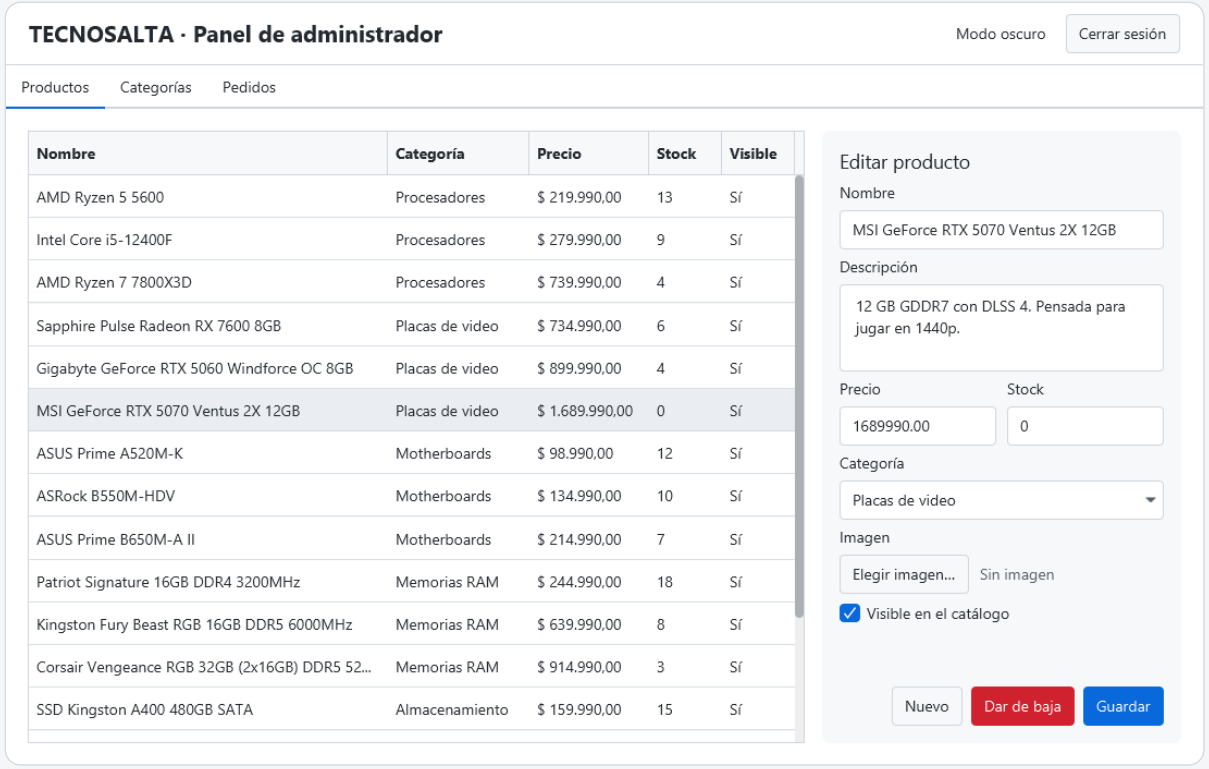
Qué hace el código

- Antes de confirmar hay una pantalla de resumen con el total
- Cada cliente ve solo sus pedidos, del más nuevo al más viejo
- Al seleccionar un pedido se muestran sus renglones
- El precio que se ve es el que se pagó, aunque después cambie

`ConfirmacionController` · `MisPedidosController` · `PedidoService`

EXPOSITOR 4 · ADMINISTRADOR

Panel de administrador



Qué hace el código

- Un TabPane con tres pestañas: Productos, Categorías y Pedidos
- Al seleccionar un renglón, sus datos pasan al formulario
- ProductoService valida nombre, precio, stock y categoría antes de guardar
- Dar de baja es un borrado lógico; cancelar un pedido devuelve el stock

`AdminController · ProductoService · PedidoService`

Pruebas automáticas

CarritoService: 5 pruebas

- Agregar el mismo producto suma la cantidad
- El total suma los subtotales
- No se puede superar el stock
- Cambiar la cantidad valida el stock
- Quitar saca el producto del carrito

PedidoService: 7 pruebas

- La compra descuenta stock y vacía el carrito
- Si falta stock, no se guarda nada
- Con el carrito vacío no se puede comprar
- El pedido conserva el precio pagado
- Cada cliente ve solo sus pedidos
- Cancelar devuelve el stock y no se reabre

Las 12 pasan con **mvnw test**. Trabajan en memoria: no leen ni modifican tienda.dat.

Planificación en siete etapas

Etapa	Qué se construyó	Listo cuando
1. Proyecto base	Maven, JavaFX y module-info	Se abre una ventana vacía
2. Modelo y datos	Las 9 clases, Almacen y los repositorios	Los datos se leen desde tienda.dat
3. Login y navegación	AuthService, Sesion y Navegador	Cada rol entra a su pantalla
4. Catálogo	Tarjetas, búsqueda y filtros	Los filtros responden al escribir
5. Carrito y compra	CarritoService, PedidoService e historial	Una compra descuenta el stock
6. Administrador	Productos, categorías y pedidos	Se crea un producto y cambia un estado
7. Pulido y entrega	Modo oscuro, pruebas, README y ejecutable	Pasan las 12 pruebas

Cada etapa termina con algo que se puede ejecutar y probar.

EXPOSITOR 5 · PRUEBAS Y CIERRE

Conclusión

Se cumplieron los objetivos: una tienda completa para dos roles, hecha solo en Java, que se abre y se muestra sin preparar nada. Diseñar antes de programar hizo que los cambios de rumbo tocaran pocas clases.

Compra todo o nada

Se verifica todo el carrito antes de modificar algo

Precio congelado

El historial muestra lo que el cliente pagó

Contraseñas cifradas

En el archivo nunca hay una clave en texto plano

Excepción propia

Los errores del negocio tienen su propia clase

Capas respetadas

Ninguna pantalla guarda datos por su cuenta

Lógica probada

12 pruebas automáticas sobre el carrito y la compra

Cómo seguir y profundizar

Próximos pasos

- Repositorio Git con un commit por etapa
- Fotos reales de los productos
- Más pruebas: login y catálogo
- Pruebas automáticas de las pantallas

Nuevas funciones

- Reportes de ventas para el administrador
- Ordenar el catálogo por precio o nombre
- Comprobante del pedido en PDF
- Instalador para Windows

Temas para profundizar

- Genéricos para no repetir código
- Streams y lambdas en los filtros
- Hilos para guardar sin trabar la ventana
- Una versión web que reutilice la lógica

TECNOSALTA · LENGUAJES II

Gracias

[Corregidor Martin Guillermo] · [Flores C. José M.] · [Fuentes Facundo] · [Tapia Lucas Fabricio] · [Camacho Daniel]